Campus Navigator: navegación de un campus universitario

1. Contexto

Quien llega por primera vez a una universidad no siempre sabe cómo ir de un lugar a otro. En este proyecto construirás **Campus Navigator**, una aplicación de consola en Java que representa un campus como un **grafo** y permite consultar sus lugares, ver qué lugares están conectados y encontrar un camino entre dos de ellos.

El campus es ficticio. No hay interfaz gráfica, base de datos, archivos externos ni Internet: todo se ejecuta en consola y los datos se escriben en el código.

2. Objetivo

Aplicar la representación de grafos con **lista de adyacencia** y los recorridos **BFS** y **DFS** para resolver un problema sencillo y concreto: encontrar una ruta entre dos lugares.

3. Qué debe hacer la aplicación

#	Funcionalidad	Qué se espera
R1	Representar el campus	Grafo **no dirigido**: cada lugar es un vértice y cada camino peatonal es una arista que se recorre en ambos sentidos. Mínimo **8 lugares y 10 conexiones**. Cada lugar tiene un número identificador (0, 1, 2...) y un nombre.
R2	Listar lugares	Muestra el identificador y el nombre de todos los lugares.
R3	Ver conexiones de un lugar	El usuario elige un lugar y se muestran los lugares conectados directamente con él. Si no tiene ninguna, lo informa.
R4	Recorrido BFS	El usuario elige un lugar de inicio y se muestra el orden en que se visitan los lugares alcanzables. Debe usar una **cola** y un registro de **visitados**.
R5	Recorrido DFS	Igual que R4, pero en

#	Funcionalidad	Qué se espera
		profundidad. Debe hacerse con **recursividad** y un registro de visitados.
R6	Ruta entre dos lugares	El usuario elige origen y destino y se muestra la secuencia de lugares para ir del uno al otro, usando **BFS** y guardando el **predecesor** de cada lugar para reconstruir el camino. Casos que debe manejar: si origen y destino son el mismo lugar, lo informa; si no hay camino, informa que el destino no es alcanzable.
R7	Menú y validación	Menú de consola con una opción por funcionalidad y una para salir. Si el usuario escribe una opción inválida o un identificador que no existe, el programa muestra un mensaje y **sigue funcionando** (no debe terminar con error).

4. Reglas técnicas

- Lenguaje: **Java**. Se pueden usar las colecciones de Java como `ArrayList`, `Queue` / `LinkedList` o `ArrayDeque`, y `Scanner`.
- El grafo se implementa con **lista de adyacencia** (por ejemplo, un `ArrayList` de listas de enteros). No se permite usar bibliotecas que ya traigan BFS, DFS o rutas.
- BFS, DFS y la ruta deben ser **implementados por el grupo de trabajo**.
- El código debe estar separado en **al menos tres clases** con responsabilidades claras. Se sugiere:
  - `Grafo`: guarda los lugares y las conexiones, con sus métodos de consulta.
  - `Algoritmos`: BFS, DFS y búsqueda de ruta.
  - `Main`: menú y lectura de datos. No debe concentrar toda la lógica.
- Debe compilar y ejecutarse en el computador local sin Internet.

5. Campus de ejemplo (puedes usar este)

Para ahorrar tiempo puedes usar este campus. Si prefieres diseñar el tuyo, debe cumplir los mínimos de R1 y tener al menos un lugar sin conexiones y un ciclo.

Id	**Lugar**	**Id**	**Lugar**

Id	Lugar	Id	Lugar
0	Entrada Principal	5	Auditorio
1	Biblioteca	6	Zona Deportiva
2	Bloque A	7	Parqueadero
3	Cafetería	8	Bodega (sin conexiones)
4	Laboratorio de Sistemas		

Conexiones (10), agrégalas en este orden:

```
0-1, 0-2, 0-3, 0-7, 1-2, 1-3, 2-4, 3-5, 4-5, 5-6
```

La Bodega queda aislada a propósito: sirve para probar «lugar sin conexiones» y «destino no alcanzable». Hay ciclos (por ejemplo 0-1-2), por lo que el registro de visitados es necesario.

6. Pruebas mínimas

Durante la demostración se ejecutan estas ocho pruebas. Con el campus de ejemplo y las conexiones agregadas en el orden indicado, los resultados esperados son:

#	Prueba	Resultado esperado
1	Listar todos los lugares	Los 9 lugares con su id y nombre
2	Conexiones de la Entrada (0)	1, 2, 3 y 7
3	Conexiones de la Bodega (8)	Informa que no tiene conexiones
4	BFS desde 0	0, 1, 2, 3, 7, 4, 5, 6
5	DFS desde 0	0, 1, 2, 4, 5, 3, 6, 7
6	Ruta 7 → 6 (Parqueadero a Zona Deportiva)	Parqueadero → Entrada Principal → Cafetería → Auditorio → Zona Deportiva
7	Ruta 0 → 8, y ruta 0 → 0	«No es alcanzable» y «la ruta es solo ese lugar»
8	Ruta o recorrido con un id inexistente (por ejemplo 99) y una opción de menú inválida	Mensaje de error y el programa continúa

Si tu resultado difiere, revisa que agregaste las conexiones en el orden indicado: el orden de los vecinos cambia el orden de visita, aunque ambos resultados serían correctos.

7. Plan sugerido de dos semanas

Cuándo	Meta	Entregable parcial
Días 1-2	Clase `Grafo` con lugares y conexiones bidireccionales; cargar el campus de ejemplo	Imprime el grafo completo
Días 3-4	R2, R3 y menú básico con validación de opciones	Menú funcionando con listar y ver conexiones
Días 5-6	R4 (BFS) con cola y visitados	BFS correcto en la prueba 4
Días 7-8	R5 (DFS recursivo)	DFS correcto en la prueba 5
Días 9-11	R6 (ruta con predecesores) y sus casos especiales	Pruebas 6 y 7 correctas
Días 12-13	Validaciones (id inexistente), pruebas 1 a 8 y limpieza del código	Las ocho pruebas pasan
Día 14	README y demostración	Entrega final

8. Pistas para empezar

- **Visitados:** un arreglo `boolean[]` del tamaño del número de lugares es suficiente.
- **BFS:** marca el inicio como visitado y ponlo en la cola. Mientras la cola no esté vacía, saca un lugar, regístralo y pon en la cola sus vecinos que no estén visitados (marcándolos al ponerlos).
- **DFS:** un método que se llama a sí mismo: marca el lugar actual, regístralo y llama al mismo método con cada vecino no visitado.
- **Ruta:** es un BFS con un arreglo `int[] predecesor`. Cuando descubras un vecino, guarda desde qué lugar llegaste. Al encontrar el destino, retrocede por `predecesor` hasta el origen y luego invierte la secuencia.
- **Validación:** antes de usar un id, verifica que esté entre 0 y el número de lugares menos 1. Para leer números, usa `nextLine()` y conviértelo con `Integer.parseInt` dentro de un `try-catch` para que una letra no cierre el programa.

9. Qué se entrega

1. El código fuente que compile y se ejecute.

2. Una **presentacion/sustentación** corta en la que explicarán todo el proceso.

En el siguiente link, hay una herramienta de IA que puede guiarlos en la elaboración del Proyecto: https://gemini.google.com/gem/1r0ndtmxmFl3tsmmrT7LjjWE9qbBkuHsg?usp=sharing

Por su puesto pueden apoyarse en otras herramientas, pero recuerden que se evaluará el conocimiento que se haya adquirido